

Rutwik Kishor Chabukswar — RAG Knowledge Base Appendix

Document Type: Supplementary Dataset — Append to Primary Knowledge Base **Purpose:** Recruiter Q&A, Behavioral Stories, Technical Architecture, Edge Cases, Semantic Variants, Follow-up Flows **Candidate:** Rutwik Kishor Chabukswar | Software Engineer | Full Stack | AI Systems

SECTION 1: Recruiter Interview Dataset

Each answer is self-contained and retrieval-ready. Metadata tag: `type: recruiter_qa`

Q: Tell me about yourself.

Rutwik Kishor Chabukswar is a final-year Information Technology engineering student at DY Patil Institute of Technology, Pune, graduating in 2026 with a CGPA of 8.16. He is a software engineer with hands-on experience in frontend development, backend API engineering, and AI/LLM systems.

His journey began with frontend development — building user interfaces with React.js and Tailwind CSS. Over time, he expanded into backend systems using Python and FastAPI, and eventually into AI-powered applications using RAG pipelines, LLM integration, and vector databases like ChromaDB.

He has completed two internships before graduation. At Hanumatrix, he built production-grade frontend applications for real client projects. At Quick Heal Technologies — a major cybersecurity company — he is currently working on internal dashboards, chatbot interfaces, Docker deployments, and enterprise IT infrastructure.

His strongest project is an AI Compliance Monitoring Agent — a real-time, RAG-based system that analyzes conversations for compliance issues. He also built SarkariSaathi, a full-stack AI platform that helps citizens query government welfare documents using natural language.

He is currently looking for full-time roles in software engineering, full stack development, or AI systems engineering.

Q: Why should we hire you?

Most engineering graduates specialize in one domain — either frontend, backend, or AI. Rutwik brings demonstrated, practical experience across all three, plus enterprise infrastructure exposure — all before graduating.

He has completed two internships: one building client-facing React applications, and one working inside an enterprise environment (Quick Heal Technologies) on dashboards, chatbots, Docker deployments, and Active Directory systems. This gives him both a product mindset and an enterprise operational perspective.

His AI projects — including the AI Compliance Monitoring Agent and SarkariSaathi — show that he can design and build end-to-end intelligent systems, not just integrate APIs. He understands RAG pipelines, retrieval optimization, LLM response generation, and backend architecture.

He is a fast learner, adaptable across domains, and capable of taking ownership of complex problems. He contributes across multiple engineering layers — which is especially valuable for smaller teams and product-focused companies.

Q: What makes you different from other candidates?

Three things make Rutwik's profile stand out from typical final-year candidates.

First, breadth of real experience: most students have either one internship or only personal projects. Rutwik has two internships — one in product development and one in enterprise engineering — plus multiple AI-powered projects built independently.

Second, AI systems depth: many students claim AI familiarity but have only used pre-built APIs. Rutwik has built full RAG pipelines — including PDF ingestion, chunking, ChromaDB vector storage, retrieval optimization, and LLM-based answer generation — from scratch.

Third, cross-domain thinking: he does not think only as a frontend developer or only as a backend engineer. He thinks in terms of complete systems — how data flows from a user interface through an API, into a processing pipeline, and back to the user. This systems-level thinking is rare at the final-year level.

Q: What kind of roles are you looking for?

Rutwik is looking for roles in:

- Software Engineering (full stack or generalist)
- Frontend Engineering (React.js / Next.js focused)
- Backend Engineering (Python / FastAPI focused)
- AI Systems Engineering (RAG, LLM integration, document intelligence)
- Product Engineering (end-to-end ownership of product features)

He is open to both startup environments and established engineering teams. He is based in Pune and open to relocation — including Mumbai and other cities depending on the role and opportunity.

Q: Why software engineering?

Rutwik was drawn to software engineering because it is one of the few fields where a single

person can build something that thousands of people use. He started with frontend development because he enjoyed seeing ideas become interactive products. That curiosity pushed him further — into backend systems to understand how data is structured and served, and then into AI systems to understand how software can reason and retrieve intelligently.

What keeps him engaged is the combination of problem-solving and creativity. Every system has a design challenge — how to structure the data, how to keep the pipeline efficient, how to make the interface feel intuitive. He finds that combination genuinely interesting, not just professionally useful.

Q: Why full stack development?

Rutwik moved toward full stack development because he wanted to own problems completely — not just implement one layer and hand off to someone else. When he was purely focused on frontend work, he often wanted to understand why certain API responses were slow, or how the backend was structuring data. When he started building backend systems, he wanted to see how the interface would present that data to users.

Full stack development gives him the ability to trace a problem from the user interface all the way to the database and back. That end-to-end perspective makes him a stronger engineer and a more useful team member — especially in smaller teams where people need to move across layers quickly.

Q: Why AI systems?

Rutwik became interested in AI systems when he realized that the most interesting engineering problems involve not just processing data, but understanding it. RAG systems, LLM integration, and document intelligence are areas where software does something genuinely useful — it helps people access information they couldn't easily access before.

SarkariSaathi is a direct example of this. Government welfare documents exist but are inaccessible to most citizens. Building a system that lets someone ask a natural language question and get an accurate answer from those documents felt like a meaningful engineering problem. That's the kind of work that keeps him motivated.

Q: Why should startups hire Rutwik?

Startups need engineers who can move fast, own problems end-to-end, and work across the stack without needing a specialist for every layer. Rutwik fits that profile well.

He has built full products independently — from designing the frontend to building backend APIs to integrating AI pipelines. He is comfortable with ambiguity and has demonstrated the ability to pick up new technologies quickly when a project requires it. His internship at Hanumatrix, a product-focused company, gave him experience working on real client projects with production expectations. He thinks in terms of user impact, not just technical implementation.

Q: Why should enterprise companies hire Rutwik?

Enterprise environments require engineers who understand how software operates inside complex organizational systems — not just how to build greenfield products. Rutwik's internship at Quick Heal Technologies gave him direct exposure to enterprise infrastructure: Active Directory, VM license management, internal dashboards, Docker deployments, and network troubleshooting.

He understands that enterprise software must be stable, maintainable, and integrated with existing systems — not just technically impressive. He has worked alongside IT infrastructure teams and understands the operational constraints that enterprise engineering teams face. That awareness makes him a more practical and effective contributor in large-scale environments.

Q: Where do you see yourself in 5 years?

In five years, Rutwik sees himself as a senior software engineer with deep expertise in one or more of: full stack systems, AI-powered products, or scalable backend infrastructure. He wants to have led meaningful engineering work — either as a technical lead on a product team or as a founding engineer at a startup.

Longer term, he is interested in building his own products — AI tools or SaaS platforms that solve real problems. He sees the skills he is building now — systems design, AI integration, full stack engineering — as the foundation for that goal.

Q: What are your strengths?

Rutwik's key strengths are:

- **Cross-domain adaptability:** He can work across frontend, backend, and AI layers — which makes him useful in a wide range of engineering contexts.
 - **Fast learning:** He has repeatedly picked up new technologies and frameworks by building with them — not just reading about them.
 - **Problem-solving under complexity:** His work at Quick Heal involved debugging systems where multiple components interacted with each other. He is comfortable tracing problems across layers.
 - **Product thinking:** He considers the end-user impact of technical decisions, not just the technical correctness.
 - **Ownership mindset:** On his personal projects, he has taken full responsibility for architecture, implementation, debugging, and iteration.
-

Q: What are your weaknesses?

Earlier in his engineering journey, Rutwik focused heavily on frontend development and underinvested in backend systems and infrastructure. He recognized this gap when he started building AI projects and realized that a strong backend architecture is what makes AI systems actually work at scale.

He addressed this directly — by learning FastAPI and Python through hands-on project work, building real backend services for SarkariSaathi and the AI Compliance Monitoring Agent, and gaining infrastructure exposure at Quick Heal Technologies. He is now comfortable across the full stack, though he continues to deepen his backend and systems design knowledge as a deliberate focus area.

SECTION 2: Behavioral Dataset (STAR Format)

Each story is self-contained. Metadata tag: `type: behavioral_star`

Story 1 — Debugging a Complex Multi-System Workflow at Quick Heal Technologies

Situation: During his internship at Quick Heal Technologies, Rutwik was working on an internal dashboard that pulled data from multiple backend services — including Active Directory, VM license management systems, and internal APIs. The dashboard was displaying incorrect or stale data for certain users, but the problem wasn't immediately visible in any single system.

Task: Rutwik needed to identify why the data was inconsistent and fix the issue without disrupting the existing systems that other teams depended on.

Action: He started by isolating each data source independently — testing the Active Directory queries, the VM license API responses, and the dashboard's own state management layer separately. He added detailed logging at each integration point to trace

where the data diverged from expectations. He discovered that a caching layer in one of the internal APIs was returning outdated records under specific conditions that only occurred for certain user account types.

He documented the root cause, proposed a targeted fix that bypassed the stale cache for those specific query conditions, and tested the fix across multiple user account scenarios before deploying.

Result: The dashboard began displaying accurate, consistent data across all user types. The fix was minimal and surgical — it didn't require changes to the underlying API or the Active Directory configuration, which kept the risk of disruption low.

Learning: Complex bugs in multi-system environments are rarely in the obvious place. Systematic isolation of each component and adding logging at integration boundaries is more effective than guessing. He also learned the importance of understanding how caching behaves under different conditions before assuming an API is returning incorrect data.

Story 2 — Solving a Frontend Performance Issue at Hanumatrix

Situation: At Hanumatrix, Rutwik was working on a client-facing React.js application that was experiencing slow rendering on lower-end devices. The application had grown to include multiple data-heavy components, and users on mobile were experiencing visible lag when navigating between sections.

Task: He needed to improve the frontend performance of the application without breaking existing functionality or requiring a major architectural rewrite — the client had a tight delivery timeline.

Action: He audited the component tree to identify unnecessary re-renders using React Developer Tools. He found several components that were re-rendering on every state change, even when their props hadn't changed. He applied React.memo to the stable

components, refactored the state management to reduce the scope of shared state, and deferred the loading of non-critical components using lazy loading and React Suspense.

He also reviewed image assets in the application, found several unoptimized images being loaded at full resolution, and replaced them with appropriately sized, compressed versions.

Result: Rendering performance improved noticeably on mobile devices. The changes reduced unnecessary re-renders significantly and brought the application within the performance targets the client expected.

Learning: Performance issues in React applications are often caused by over-broad state and unnecessary re-renders rather than inherently slow code. Profiling before optimizing is essential — he would have wasted time optimizing the wrong components without first measuring where the actual bottleneck was.

Story 3 — Improving Retrieval Accuracy in SarkariSaathi

Situation: While building SarkariSaathi — the AI-powered government scheme assistant — Rutwik ran into a significant retrieval accuracy problem. The RAG pipeline was failing to return the most relevant sections from government PDF documents when users asked specific questions. The retrieved chunks often contained tangentially related text instead of the precise answer.

Task: He needed to improve retrieval precision without changing the underlying LLM, since the generation quality was fine — the problem was in what content was being retrieved and passed to the model.

Action: He analyzed the chunks being retrieved for a set of test queries and identified two root causes. First, the PDF documents were being chunked at fixed character intervals, which was splitting semantically related sentences across chunk boundaries. Second, the chunk size was too large — the retriever was pulling broad sections that diluted the relevance signal.

He rewrote the chunking logic to split on paragraph and section boundaries instead of character count, and reduced the chunk size to keep each chunk focused on a single coherent idea. He also added overlapping context windows between adjacent chunks to prevent information loss at boundaries. After re-embedding and re-indexing the documents in ChromaDB, retrieval accuracy improved substantially on the same test queries.

Result: The retrieved chunks became tighter and more topically focused, which directly improved the quality of the final answers generated by the LLM. The system went from occasionally returning incorrect or irrelevant answers to consistently retrieving the right document sections.

Learning: In RAG systems, retrieval quality is the most important variable — a better LLM cannot compensate for poor retrieval. Chunking strategy is not a default setting; it needs to be designed for the specific structure and content of the documents being indexed.

Story 4 — Designing Modular Architecture in the AI Compliance Monitoring Agent

Situation: When Rutwik started building the AI Compliance Monitoring Agent, his initial pipeline design was a single, sequential flow: input → process → output. This worked for small test cases but became a problem as the processing logic grew more complex. Adding a new compliance rule or changing the retrieval strategy required touching multiple parts of the pipeline simultaneously.

Task: He needed to redesign the architecture so that individual pipeline stages could be modified, tested, and scaled independently — without requiring changes to adjacent components.

Action: He refactored the pipeline into discrete, independently callable modules: an ingestion module, a retrieval module, a context assembly module, and a response generation module. Each module had a clearly defined input/output contract. He used FastAPI's dependency injection pattern to wire the modules together, which made it easy to

swap out individual components (for example, replacing the retrieval strategy) without touching the rest of the pipeline.

He also added intermediate validation checkpoints between modules so that errors could be caught and logged at the boundary where they occurred rather than surfacing as cryptic failures at the end of the pipeline.

Result: The modular architecture made the system significantly easier to iterate on. Adding a new compliance rule required only changes to the retrieval and context assembly modules, not the full pipeline. Debugging became faster because errors were localized to specific module boundaries.

Learning: Designing for modularity upfront costs some initial time but pays off quickly as a system grows in complexity. In AI pipelines specifically, the ability to swap retrieval strategies or generation logic without rewriting the whole system is critical for experimentation and improvement.

Story 5 — Learning Docker Quickly at Quick Heal Technologies

Situation: When Rutwik joined Quick Heal Technologies as an IT Engineering Intern, he was expected to assist with Docker-based deployment workflows. He had theoretical familiarity with Docker from his studies but had not worked with it in a real enterprise deployment context.

Task: He needed to get up to speed on Docker quickly enough to contribute meaningfully to deployment support within the first few weeks — without slowing down the team.

Action: Rather than spending time on tutorials alone, he asked his team lead to walk him through one real deployment from start to finish, taking detailed notes on each step and the reasoning behind it. He then replicated a similar deployment in a sandboxed environment on his own, deliberately introducing small errors to understand what the error outputs meant and how to fix them. He also read through the Dockerfiles and docker-compose

configurations used by the team to understand how the application environment was constructed.

Within two weeks, he was able to assist with deployment tasks independently, identify configuration issues in existing Dockerfiles, and help debug container networking problems.

Result: He contributed to Docker-related tasks within the internship timeline without requiring ongoing hand-holding from senior team members. He also documented the deployment process for a specific internal tool, which became a reference for other interns joining the team.

Learning: The fastest way to learn an infrastructure tool is to use it in a real context, not a toy example. Deliberately breaking things in a safe environment and understanding the error messages is more effective than reading documentation in isolation. He also learned the value of documenting processes as he learns them — it reinforces understanding and creates team value simultaneously.

SECTION 3: Deep Technical Architecture Dataset

Each architecture is self-contained and retrieval-ready. Metadata tag: `type: technical_architecture`

Architecture 1 — AI Compliance Monitoring Agent

System Overview

An AI-powered backend system that ingests conversation data, analyzes it for compliance-relevant patterns, and generates structured compliance assessment outputs in real time. Built with Python, FastAPI, RAG, LLM integration, and streaming pipelines.

User Input Flow

1. A compliance officer or automated system submits a conversation transcript (text) via a POST request to the FastAPI backend.
2. The input is validated and preprocessed — cleaned, normalized, and formatted for downstream processing.

Backend Flow

1. The FastAPI backend receives the validated input and routes it to the ingestion module.
2. The ingestion module passes the conversation to the retrieval module.

Processing Flow (RAG Pipeline)

1. **Retrieval Module:** The conversation text is embedded using a text embedding model. The resulting vector is used to query the vector store for the most relevant compliance rules, regulatory guidelines, or past compliance examples.
2. **Context Assembly Module:** Retrieved chunks are assembled into a structured context window, ordered by relevance score.
3. **Response Generation Module:** The assembled context and the original conversation are passed to the LLM with a structured prompt. The LLM generates a compliance assessment — flagging specific segments, categorizing issue types, and producing a summary.

Output Flow

1. The compliance assessment is returned to the caller as a structured JSON response.
2. The response includes: flagged segments, compliance categories, severity ratings, and a natural language summary.

3. For streaming configurations, partial results are streamed back incrementally as the LLM generates output.

Scalability Design

- Each pipeline module (ingestion, retrieval, context assembly, generation) is independently callable and stateless.
- FastAPI's async capabilities allow concurrent processing of multiple conversation inputs.
- The vector store can be scaled horizontally as the compliance rule database grows.
- New compliance rules can be added by re-indexing new documents without changing application code.

Failure Handling

- Intermediate validation checkpoints between modules catch and log errors at the boundary where they occur.
- If retrieval returns zero relevant chunks above a confidence threshold, the system returns a structured fallback response indicating low-confidence output rather than a silent failure.
- FastAPI exception handlers ensure all errors return structured JSON error responses, not raw stack traces.

Technical Tradeoffs

- **RAG vs fine-tuning:** RAG was chosen over fine-tuning because compliance rules change frequently. RAG allows the knowledge base to be updated without retraining the model.
- **Streaming vs batch:** Streaming was implemented to provide faster perceived response times for long conversations, at the cost of slightly more complex client-side handling.

- **Modular pipeline vs monolithic:** Modular design adds initial setup complexity but makes testing, debugging, and iteration significantly easier as the system evolves.

Future Improvements

- Add a feedback loop where compliance officers can flag incorrect assessments, which feeds into retrieval re-ranking.
 - Implement multi-document compliance rule sets with source attribution in the output.
 - Add a dashboard layer for visualizing compliance trends over time across multiple conversations.
-

Architecture 2 — SarkariSaathi (AI-Powered Government Scheme Assistant)

System Overview

A full-stack AI platform that allows citizens to ask natural language questions about government welfare schemes and receive accurate answers sourced directly from official government PDF documents. Built with React.js (frontend), FastAPI (backend), ChromaDB (vector store), PyPDF2 (document extraction), and LLM integration.

User Input Flow

1. A user types a natural language question into the React.js frontend (e.g., "Am I eligible for PM Kisan Samman Nidhi?").
2. The frontend sends the question as a POST request to the FastAPI backend.

Frontend Flow

1. The React.js interface renders the user's question and displays a loading state while the backend processes the query.

2. When the response arrives, the answer is rendered alongside source attribution (which document section the answer came from).
3. The interface is designed for accessibility — simple, mobile-responsive, and usable by citizens with low digital literacy.

Backend Flow

1. FastAPI receives the query and passes it to the retrieval pipeline.
2. The query is embedded using a text embedding model to produce a query vector.

Document Ingestion Flow (Offline / Setup Phase)

1. Government PDF documents are loaded using PyPDF2.
2. Text is extracted and split into semantically coherent chunks — split on paragraph and section boundaries, not fixed character counts.
3. Overlapping context windows are applied between adjacent chunks to prevent information loss at boundaries.
4. Each chunk is embedded and stored in ChromaDB with metadata (source document, page number, section title).

Retrieval Flow

1. The query vector is compared against all stored chunk embeddings in ChromaDB using cosine similarity.
2. The top-k most relevant chunks are retrieved along with their similarity scores.
3. Chunks below a minimum relevance threshold are filtered out before being passed to the LLM.

Output Flow

1. The retrieved chunks and the original query are assembled into a structured prompt.

2. The LLM generates a precise, citizen-friendly answer based only on the retrieved content.
3. The answer is returned to the React.js frontend with source references.

Scalability Design

- ChromaDB can be extended to index additional government documents without changing the application logic.
- The chunking pipeline is reusable — new PDFs can be ingested by running the ingestion script against new files.
- The FastAPI backend is stateless and can be horizontally scaled behind a load balancer.

Failure Handling

- If no chunks above the relevance threshold are retrieved, the system returns a graceful response indicating that the specific information was not found in the current document set, rather than hallucinating an answer.
- PyPDF2 extraction errors are caught and logged per document — a single malformed PDF does not break the ingestion pipeline for other documents.

Technical Tradeoffs

- **ChromaDB vs hosted vector DB:** ChromaDB was chosen for its ease of local deployment and open-source availability. For production at scale, a hosted vector database (e.g., Pinecone) would offer better performance and managed infrastructure.
- **Paragraph-based chunking vs fixed-size chunking:** Paragraph-based chunking preserves semantic coherence at the cost of variable chunk sizes. This tradeoff was deliberately made because consistent chunk relevance was more important than uniform embedding dimensions in this use case.

- **Source attribution:** Including source document references in the output was a deliberate design decision to allow users to verify answers independently — important for government information.

Future Improvements

- Multi-language support (Hindi, Marathi, and other regional languages) to improve accessibility for rural users.
 - Conversational memory — allowing users to ask follow-up questions within a session without re-embedding the query from scratch.
 - Admin interface for document management — adding, removing, and updating indexed PDFs through a UI rather than CLI scripts.
-

Architecture 3 — Active Directory User Dashboard + Chatbot (Quick Heal Technologies)

System Overview

An internal enterprise tool built during the Quick Heal Technologies internship. Consists of a React-based dashboard for visualizing Active Directory user data and VM license information, combined with a chatbot interface for querying internal operational data through natural language. Built with React, JavaScript, FastAPI, and internal API integrations.

User Input Flow (Dashboard)

1. IT administrators access the dashboard through an internal web interface.
2. The dashboard fetches user account data and license status from backend APIs connected to Active Directory and VM management systems.

3. Data is displayed in structured tables and summary cards — showing user status, license assignments, and operational alerts.

User Input Flow (Chatbot)

1. IT staff type operational queries into the chatbot interface (e.g., "How many VM licenses are currently active?").
2. The chatbot frontend sends the query to the FastAPI backend.

Backend Flow

1. The FastAPI backend routes dashboard requests to the appropriate internal data sources — Active Directory queries for user data, VM management APIs for license data.
2. For chatbot queries, the backend processes the natural language input and maps it to a predefined query pattern or API call against the internal data sources.
3. Responses are formatted and returned to the frontend.

Data Integration Flow

1. Active Directory is queried through appropriate internal APIs or directory service connectors.
2. VM license data is pulled from the VM management system's API.
3. Both data sources are normalized into a consistent format before being served to the frontend.

Output Flow

1. Dashboard: structured tables, summary statistics, and status indicators are rendered in the React interface.

2. Chatbot: natural language answers are generated from the query results and displayed in the chat interface.

Scalability Design

- The modular API design allows new internal data sources to be connected to the dashboard without redesigning the frontend.
- The chatbot's query-to-API mapping layer can be extended with new query patterns as internal tooling needs evolve.

Failure Handling

- Stale or cached data from internal APIs is handled by adding cache-busting logic for queries where data freshness is critical.
- API connection failures surface as structured error states in the dashboard rather than blank or broken UI components.

Technical Tradeoffs

- **Internal API integration vs direct system access:** Routing all queries through a FastAPI backend layer — rather than querying Active Directory directly from the frontend — adds a small latency cost but keeps credentials and system access contained on the backend.
- **Predefined chatbot patterns vs full NLU:** Given the internal, constrained use case, mapping natural language queries to predefined API calls was more reliable and predictable than deploying a full NLU model for operational queries.

Future Improvements

- Role-based access control — restricting which dashboard sections and data are visible based on the logged-in user's IT role.

- Alerting integration — triggering notifications when license counts drop below a threshold or user account anomalies are detected.
 - Audit log viewer — displaying a history of Active Directory changes directly within the dashboard.
-

SECTION 4: Edge Case / Fallback Dataset

Each response is graceful, professional, and retrieval-ready. Metadata tag: `type: edge_case_fallback`

Salary Expectations

Rutwik is open to discussing compensation based on the role, responsibilities, and the company. He is looking for a package that is competitive for a fresher software engineer in India with his experience profile — including two internships and multiple AI and full-stack projects. He is happy to discuss specifics directly with the hiring team at the appropriate stage of the process.

Joining Timeline / Availability

Rutwik is completing his Bachelor of Engineering in 2026. He is available to join full-time from mid-2026 onward, aligned with his graduation. He is open to discussing specific start dates with potential employers and can be flexible depending on the role and onboarding timeline.

Relocation

Rutwik is currently based in Pune, Maharashtra, India. He is open to relocation for the right opportunity — including Mumbai and other cities. He is willing to discuss relocation requirements and timelines with potential employers on a case-by-case basis.

Notice Period

As a final-year student, Rutwik does not have a formal notice period for his internship. He is available to begin discussions and processes immediately, and can commit to a start date aligned with his graduation timeline.

Confidential Company Details

Questions about specific internal systems, client names, internal tools, or proprietary processes from Rutwik's internships at Hanumatrix and Quick Heal Technologies cannot be answered here, as those details are confidential to those organizations. Rutwik can speak generally about the types of systems he worked on, the technologies he used, and the engineering challenges he solved — without disclosing company-confidential information.

Personal Contact Details

Specific personal contact information — phone number, personal email, or home address — is not included in this knowledge base. Recruiters and hiring teams who wish to contact Rutwik directly can reach out through the portfolio website's contact section or through LinkedIn.

Questions About Technologies Rutwik Has Not Used

Rutwik's current technology experience spans React.js, Next.js, JavaScript, Python, FastAPI, RAG pipelines, ChromaDB, PyPDF2, Docker (exposure), and Active Directory (exposure). For technologies outside this set — for example, Go, Rust, Kubernetes, GraphQL, or specific cloud platforms like AWS or GCP — Rutwik does not currently have hands-on production experience. He has demonstrated a strong pattern of picking up new technologies quickly through project-based learning, and is open to discussing his capacity to learn specific technologies required for a role.

Questions Outside His Area of Expertise

If a question falls outside Rutwik's documented experience — for example, questions about embedded systems, hardware engineering, data science modeling, or highly specialized DevOps infrastructure — the honest answer is that those areas are outside his current expertise. He is a software engineer focused on full stack development and AI systems, not a generalist across all of engineering. He is transparent about the boundaries of his knowledge and focuses on deepening the areas he is already building in.

SECTION 5: Semantic Search Optimization Dataset

Alternate phrasings, synonyms, and keyword variants for retrieval coverage. Metadata tag: `type: semantic_variants`

Frontend Development

Primary terms: frontend, front-end, front end, UI development, user interface development, client-side development **Technologies:** React, React.js, ReactJS, Next.js,

NextJS, JavaScript, JS, ES6, HTML, HTML5, CSS, CSS3, Tailwind, Tailwind CSS, responsive design, component-based development, SPA, single-page application **Role variants:** frontend engineer, frontend developer, UI engineer, web developer, React developer, JavaScript developer **Task variants:** building interfaces, building UI, building web apps, building components, making responsive layouts, cross-browser compatibility, performance optimization, reusable components

Backend Development

Primary terms: backend, back-end, back end, server-side development, API development **Technologies:** Python, FastAPI, REST API, RESTful API, HTTP API, web services, API integration, backend services **Role variants:** backend engineer, backend developer, Python developer, API developer, server-side engineer **Task variants:** building APIs, designing APIs, building backend services, designing backend architecture, handling data workflows, service integration, debugging backend, system design

FastAPI

Primary terms: FastAPI, Fast API, Python web framework, Python API framework **Variants:** async Python, Python REST framework, Python backend framework, Python server **Context:** FastAPI is the Python backend framework Rutwik used to build backend APIs for SarkariSaathi and the AI Compliance Monitoring Agent

React / React.js

Primary terms: React, React.js, ReactJS, React framework **Variants:** component library, frontend framework, JavaScript UI library, React components, React hooks, functional

components, useState, useEffect **Context:** React.js is the primary frontend technology Rutwik used across both internships and personal projects

RAG (Retrieval-Augmented Generation)

Primary terms: RAG, Retrieval-Augmented Generation, retrieval augmented generation

Variants: document retrieval, semantic search, vector search, embedding-based retrieval, knowledge retrieval, context retrieval, intelligent search, AI search, document Q&A, document question answering, PDF Q&A **Related:** vector database, embeddings, chunk retrieval, top-k retrieval, cosine similarity, context window

AI Systems

Primary terms: AI systems, artificial intelligence systems, intelligent systems, AI-powered applications **Variants:** LLM integration, large language model integration, AI chatbot, AI assistant, generative AI, NLP systems, natural language processing, document intelligence, conversational AI, AI pipeline, machine learning application **Context:** Rutwik has built two AI-powered systems: AI Compliance Monitoring Agent and SarkariSaathi

Docker

Primary terms: Docker, containerization, container deployment **Variants:** Docker containers, Dockerfiles, docker-compose, container orchestration, deployment containers, containerized applications **Context:** Rutwik gained Docker exposure during his internship at Quick Heal Technologies — assisting with deployment pipelines and debugging container configurations

Primary terms: internship, internships, work experience, industry experience **Variants:** intern, industrial training, corporate experience, professional experience, workplace experience, on-the-job training **Entities:** Hanumatrix internship, Quick Heal Technologies internship, frontend internship, IT engineering internship, enterprise internship, product development internship

Projects

Primary terms: projects, portfolio projects, personal projects, engineering projects, software projects **Variants:** side projects, independent projects, academic projects, AI projects, full stack projects, open-source projects, portfolio **Entities:** AI Compliance Monitoring Agent, SarkariSaathi, Mission Archive, Carwash Platform, AD User Dashboard

Enterprise Systems

Primary terms: enterprise systems, enterprise software, enterprise applications, internal tools **Variants:** internal dashboards, operational tools, IT infrastructure, enterprise IT, corporate systems, enterprise engineering, Active Directory, VM management, license tracking, Windows administration **Context:** Rutwik built enterprise tools during his Quick Heal Technologies internship

Debugging

Primary terms: debugging, debug, troubleshooting, bug fixing, issue resolution **Variants:** root cause analysis, fixing bugs, diagnosing issues, tracing errors, logging, error handling, system debugging, workflow debugging, multi-system debugging **Context:** Rutwik has

debugging experience across frontend (React), backend (FastAPI/Python), AI pipelines (RAG), and enterprise systems (Active Directory, internal APIs)

Full Stack Development

Primary terms: full stack, full-stack, full stack development, full stack engineering

Variants: end-to-end development, frontend + backend, complete product development, web application development, software engineering generalist **Context:** Rutwik is a full stack developer with experience in React.js (frontend), Python/FastAPI (backend), ChromaDB (data layer), and LLM integration (AI layer) — demonstrated through SarkariSaathi and the AI Compliance Monitoring Agent

SECTION 6: Follow-up Question Dataset

Each follow-up chain is self-contained. Metadata tag: `type: followup_qa`

Follow-ups After Project Explanations

After: "Tell me about SarkariSaathi"

Follow-up 1: How does the RAG pipeline work in SarkariSaathi? The RAG pipeline in SarkariSaathi works in two phases. In the offline ingestion phase, government PDF documents are processed using PyPDF2 to extract text, which is then split into semantically coherent chunks (split at paragraph boundaries with overlapping context windows). Each chunk is embedded using a text embedding model and stored in ChromaDB with metadata including the source document and section. In the online retrieval phase, when a user submits a question, the question is embedded and compared against stored chunk

embeddings using cosine similarity. The top-k most relevant chunks are retrieved, assembled into a context prompt, and passed to an LLM which generates the final answer.

Follow-up 2: What vector database did you use and why? ChromaDB was used as the vector database for SarkariSaathi. It was chosen because it is open-source, easy to run locally without cloud infrastructure, and integrates cleanly with Python-based RAG pipelines. For a production deployment at scale, a hosted vector database like Pinecone would offer better performance and managed infrastructure — but ChromaDB was the right choice for the project's current stage.

Follow-up 3: How did you handle cases where the answer wasn't in the documents? If no chunks above a defined relevance threshold are retrieved from ChromaDB, the system is designed to return a graceful response indicating that the specific information was not found in the current document set — rather than passing low-relevance chunks to the LLM and risk generating a hallucinated answer. This threshold-based filtering is a deliberate design decision to maintain answer reliability, especially important for government information.

Follow-up 4: Could SarkariSaathi support multiple languages? Multi-language support is a planned future improvement. The current implementation processes documents in English, but the architecture supports adding Hindi and Marathi language documents to the index. The main requirement would be using an embedding model that supports multilingual text — models like `paraphrase-multilingual-mpnet` are designed for this. The chunking and retrieval pipeline would not need significant changes.

After: "Tell me about the AI Compliance Monitoring Agent"

Follow-up 1: How does real-time processing work in this system? Real-time processing in the AI Compliance Monitoring Agent is implemented through FastAPI's async capabilities combined with streaming output from the LLM. When a conversation transcript is submitted, the modular pipeline processes it asynchronously — ingestion, retrieval,

context assembly, and generation each run as discrete async operations. For streaming configurations, the LLM's output is streamed back to the caller incrementally as it is generated, reducing perceived latency for long conversations.

Follow-up 2: What compliance rules does it check? The compliance rules are stored as documents in the vector store — not hardcoded into the application logic. This means the system checks for whatever rules have been indexed. The RAG approach allows compliance rule sets to be updated by adding new documents to the index without changing the application code. This is one of the key architectural advantages of using RAG for compliance monitoring rather than rule-based matching.

Follow-up 3: How do you ensure the system doesn't hallucinate compliance issues? The system uses retrieved context as the grounding for all assessments — the LLM is prompted to base its output only on the retrieved compliance rules and the submitted conversation. Chunks below a minimum relevance threshold are filtered out before reaching the LLM. Additionally, the output is structured to include source references — indicating which compliance rules informed each flagged segment — which makes the output verifiable rather than opaque.

Follow-up 4: How would this scale to high conversation volumes? The modular, stateless pipeline architecture is designed with horizontal scaling in mind. Each FastAPI instance processes requests independently, so multiple instances can run behind a load balancer to handle concurrent conversation inputs. The vector store can be scaled or replaced with a hosted solution as the rule database grows. For very high volumes, the ingestion and generation steps could be decoupled using an async message queue (e.g., Redis or Kafka), allowing ingestion to proceed independently of generation latency.

Follow-ups After Internship Explanations

After: "Tell me about your internship at Quick Heal"

Follow-up 1: What kind of internal tools did you build at Quick Heal? At Quick Heal Technologies, Rutwik built an internal dashboard for IT administrators that displayed Active Directory user data and VM license status. He also built a chatbot interface integrated with the dashboard that allowed IT staff to query operational data using natural language. Both tools were designed to reduce the manual effort involved in monitoring internal IT systems.

Follow-up 2: What was working with Active Directory like? Working with Active Directory involved understanding how enterprise user accounts, groups, and permissions are structured in a Windows environment. Rutwik worked with internal APIs that interfaced with Active Directory to pull user account data — not directly with the AD protocol itself. The challenge was handling the variety of account states (active, disabled, locked) and ensuring the dashboard represented them accurately and in real time.

Follow-up 3: Is Quick Heal Technologies a large company? Yes, Quick Heal Technologies is a well-established Indian cybersecurity company. Working there gave Rutwik exposure to an enterprise engineering environment — which is meaningfully different from a startup. Enterprise environments have more complex internal systems, stricter operational requirements, and more interdependent teams. That exposure shaped how Rutwik thinks about building software that needs to integrate with existing organizational infrastructure.

After: "Tell me about your internship at Hanumatrix"

Follow-up 1: What kind of clients did Hanumatrix work with? Hanumatrix is a product development company. The specific clients and projects Rutwik worked on are confidential to the organization. Generally, the work involved building client-facing web applications — responsive, performant React.js frontends integrated with backend APIs — with real production expectations and delivery timelines.

Follow-up 2: Did you work alone or in a team at Hanumatrix? Rutwik worked as part of a development team at Hanumatrix. This involved code collaboration using Git, coordinating on component design with other developers, and communicating with the team lead on delivery priorities. Working in a team environment on real client projects was one of the key professional learnings from that internship.

Follow-ups After Skill Discussions

After: "What backend technologies do you know?"

Follow-up 1: Have you worked with databases? Rutwik has worked with ChromaDB as a vector database for the RAG systems in SarkariSaathi and the AI Compliance Monitoring Agent. He has also worked with data from relational-style sources in the context of internal API integrations at Quick Heal. His primary backend data experience is with vector stores and API-integrated data sources rather than traditional SQL databases — though he is familiar with SQL concepts and is actively building in that direction.

Follow-up 2: Do you know any cloud platforms — AWS, GCP, Azure? Rutwik does not currently have hands-on production experience with major cloud platforms such as AWS, GCP, or Azure. His deployment experience has been with Docker-based local and on-premise configurations. Cloud platform experience is an area he is actively looking to develop in his next role.

Follow-up 3: Can you explain what FastAPI is and why you chose it? FastAPI is a modern Python web framework for building REST APIs. It is built on top of Starlette for the web layer and Pydantic for data validation, and it supports async request handling natively. Rutwik chose FastAPI for his backend projects because it is fast to develop with, produces clean and well-validated API endpoints, generates automatic API documentation via

Swagger UI, and handles async operations efficiently — which is important for AI pipelines where LLM calls can be slow.

Follow-ups After Hiring Questions

After: "Why should we hire you?"

Follow-up 1: Can you give a specific example of your cross-domain contribution?

SarkariSaathi is the clearest example. Rutwik owned the project end-to-end: he designed and built the React.js frontend for the user interface, built the FastAPI backend for handling queries, designed the RAG pipeline for document retrieval using ChromaDB and PyPDF2, and integrated the LLM for answer generation. No layer was built by someone else. That end-to-end ownership — across frontend, backend, and AI — in a single project demonstrates what cross-domain contribution looks like in practice.

Follow-up 2: What's the most complex system you've built independently? The AI Compliance Monitoring Agent is the most architecturally complex system Rutwik has built independently. It involves real-time conversation ingestion, a multi-stage RAG pipeline with modular components, LLM-based assessment generation, streaming output, and structured error handling at each pipeline boundary. Designing the modular architecture — so that each stage could be tested and modified independently — required thinking about system design beyond just making the code work.

Follow-up 3: How do you handle working under pressure or tight deadlines? At Hanumatrix, Rutwik worked on client-facing applications with real delivery timelines. When a frontend performance issue arose close to a delivery window, he had to prioritize, diagnose quickly using React Developer Tools, apply targeted fixes (React.memo, lazy loading, image optimization), and validate the changes without introducing regressions —

all within the available timeline. He focuses on structured diagnosis rather than reactive changes under pressure, which keeps fixes reliable even when time is constrained.

Suggested Chunking Strategy (Appendix Supplement)

New Chunks to Add

Chunk	Section	Type Tag
16	Recruiter Q&A — Tell me about yourself	recruiter_qa
17	Recruiter Q&A — Why hire you / What makes you different	recruiter_qa
18	Recruiter Q&A — Role interests / Why software / Why full stack / Why AI	recruiter_qa
19	Recruiter Q&A — Startup fit / Enterprise fit	recruiter_qa
20	Recruiter Q&A — Future goals / Strengths / Weaknesses	recruiter_qa
21	STAR — Debugging at Quick Heal	behavioral_star
22	STAR — Performance optimization at Hanumatrix	behavioral_star
23	STAR — Retrieval accuracy in SarkariSaathi	behavioral_star
24	STAR — Modular architecture in AI Compliance Agent	behavioral_star
25	STAR — Learning Docker quickly	behavioral_star
26	Architecture — AI Compliance Monitoring Agent	technical_architecture
27	Architecture — SarkariSaathi	technical_architecture

Chunk	Section	Type Tag
28	Architecture — AD Dashboard + Chatbot	technical_architecture
29	Edge Cases — Salary / Timeline / Relocation / Confidentiality	edge_case_fallback
30	Edge Cases — Unknown technologies / Outside expertise	edge_case_fallback
31	Semantic Variants — Frontend / Backend / React / FastAPI	semantic_variants
32	Semantic Variants — RAG / AI Systems / Docker / Full Stack	semantic_variants
33	Follow-ups — SarkariSaathi project	followup_qa
34	Follow-ups — AI Compliance Agent project	followup_qa
35	Follow-ups — Quick Heal internship	followup_qa
36	Follow-ups — Hanumatrix internship	followup_qa
37	Follow-ups — Skills / Backend / Cloud / FastAPI	followup_qa
38	Follow-ups — Hiring questions / Cross-domain / Pressure	followup_qa

Updated Metadata Schema

json

```
{  
  "candidate": "Rutwik Kishor Chabukswar",  
  "type": "recruiter_qa | behavioral_star | technical_architecture | edge_case_  
  "domain": ["frontend", "backend", "AI", "infrastructure", "full-stack", "ente  
  "entity": "Hanumatrix | Quick Heal Technologies | SarkariSaathi | AI Complian  
  "technologies": ["React.js", "Python", "FastAPI", "RAG", "ChromaDB", "PyPDF2"  
  "keywords": ["software engineer", "full stack", "LLM", "internship", "RAG", "  
}
```